

Built-in tools inside agents

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 12 of 12 · English notes

Session 12 (27 August 2026) finished **built-in tool use** on the Claude Agent SDK. You do not write Read/Write/Edit/Bash yourselves. You *allow* them on the agent, give a goal, and let Claude Code execute while Claude decides. Materials were pushed to the cohort GitHub; yesterday's PDF was still uploading at the start of the call.

Why built-ins exist

Module 10 custom tools were clocks and HTTP wrappers. Agents that work on a repo need filesystem and shell primitives the SDK already ships. Enabling a tool is a permission decision as much as a feature flag — module 12 chapter 02 treats that as safety. This chapter is capability.

NAMES DRIFT

Exact tool identifiers (Bash vs Shell, Monitor vs a streaming helper) must be checked on current **Claude Code** and Agent SDK docs. The pedagogy below follows the 27 August notebook.

Bash versus Monitor

Allowing **Bash** lets the agent run scripts, OS commands, and Git. The demo prompt asked for a system report: Python version (the trainer's machine showed 3.13-class), cwd, disk space, date — presented as a readable report. Claude chose the commands; Bash ran them.

Monitor was taught as the partner for long or streaming work: a script in the background, each stdout line fed to Claude as it appears, instead of one blob when the process exits. Short one-or-two commands: Bash alone is enough. A series of steps where a permission error should be visible *mid-run*: enable Monitor with Bash. The build_script.py demo printed a line every three seconds; the agent was told to react in one sentence per line before the next arrived.

Who writes the retry loop?

Mritunjay wanted to see an explicit “if error then edit” loop in Python. Vijay's answer: **the model owns the loop** inside query's message iterator. You enable Monitor, Bash, Read, and Edit; you write a prompt that says “if it fails, read the file, explain the bug, edit, re-monitor.” The flaky_build.py demo deliberately divided by zero; after path confusion (relative vs absolute, wrong working directory) the lesson stood: prompts plus allowed tools, not a hand-coded state machine, drove the recover-and-rerun behaviour.

Glob versus Grep

These are the search tools Claude Code uses when it surveys a project before editing.

- **Glob** — find files by name/extension over a tree (“all .py under search_demo, full paths, count them”).

- **Grep** — find lines inside files (TODO comments, `^def` function headers). Regex is expected.

A combined goal: Glob all Python files, Grep their function definitions, emit a clean report. That snapshot is context for later Write/Edit — the same reason you should not dump an entire monorepo into a prompt (RAG logic, filesystem edition).

Web search versus Web fetch

Tool	Trainer analogy	What returns to Claude
Web search	Google-like query	A synthesised hunt over live web results (not the model's training cutoff)
Web fetch	Scrape a URL	The page (or file) at a known address, downloaded for extraction

Without search, Claude answers from parameters that may be months old. With search, the *client* tool acts as a browser, then the *server* (LLM) formats. Trade-off: search results become **input tokens**. Be specific or you buy a large download. Fetch is cheaper when you already have the URL. The demo searched for Claude Agent SDK Python examples and wrote notes toward Markdown.

Ask user question (human in the loop)

When the goal needs a human choice (output format, multi-select vs single), enable **AskUserQuestion** and a `can_use_tool` callback. The notebook stored questions as JSON (options, whether multi-select). The agent pauses, your callback collects the answer, the loop continues. That is the same family as permission prompts, but for product questions rather than “may I run rm?”

Exam angle

If the scenario is “latest docs for the Agent SDK,” pick web search (or fetch a known docs URL). If it is “list every TODO in src/,” pick Grep (and maybe Glob first). If it is “run a long build and comment as logs appear,” pick Monitor+Bash. If it is “current Python version on this laptop,” pick Bash, not a custom clock tool.

Official documentation

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Built-in tools the terminal agent uses.

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Custom tools versus SDK built-ins — same handshake.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Third-party tools sit beside built-ins on the allow list.

- **Rate limits**

<https://docs.claude.com/en/api/rate-limits>

Web search loops can be expensive.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.